

Portfolio personale dinamico

Proposta progettuale aggiornata con backend Node.js, MySQL e area admin JWT - Programmazione Web e Mobile

Docente: Carlo Taticchi - Università degli Studi di Perugia

Il progetto consiste in un portfolio personale dinamico composto da pagine pubbliche, area amministrativa protetta, backend Express e database MySQL. L'obiettivo è dimostrare una applicazione completa, coerente con gli argomenti del corso e realizzata senza framework frontend non richiesti.

1. Obiettivi del progetto

- Realizzare un sito personale responsive con almeno cinque pagine HTML pubbliche.
- Separare struttura, stile e comportamento tramite HTML, CSS unico e JavaScript vanilla.
- Caricare progetti e competenze da API REST collegate a MySQL.
- Salvare i messaggi del form contatti nel database.
- Permettere all'amministratore di gestire i progetti tramite CRUD protetto da JWT.
- Applicare prepared statement, validazione input e cookie HttpOnly.

2. Tecnologie utilizzate

Livello	Tecnologia	Uso nel progetto
Frontend	HTML5	Struttura semantica di pagine, form e navigazione
Frontend	CSS3	Unico foglio stile, responsive design, tema chiaro/scuro
Frontend	JavaScript	DOM, eventi, fetch, validazione e area admin
Backend	Node.js + Express	Server HTTP, API REST, middleware, file statici
Database	MySQL	Persistenza di progetti, skills, messaggi e admin
Sicurezza	JWT + cookie HttpOnly	Autenticazione admin senza localStorage per il token

3. Pagine previste

Pagina	Funzione
index.html	Home con presentazione, call to action e progetti in evidenza
about.html	Profilo, percorso, timeline e competenze dinamiche
projects.html	Elenco progetti con ricerca, filtro, ordinamento e conteggio
project.html	Dettaglio progetto letto da id URL e API
contact.html	Contatti e form con salvataggio messaggio su MySQL
404.html	Pagina di errore personalizzata
admin-login.html	Login amministratore
admin-dashboard.html	Dashboard protetta per gestione progetti
admin-project-form.html	Form protetto per inserimento o modifica progetto

4. Architettura frontend

Il frontend è contenuto nella cartella public. Ogni pagina mantiene la struttura header, nav, main e footer. Il body usa un attributo data-page che permette ad app.js di inizializzare solo la logica della pagina corrente.

- Menu responsive con pulsante burger su smartphone.
- Tema chiaro/scuro gestito con variabili CSS e preferenza in localStorage.
- Card progetto create dinamicamente a partire dai dati ricevuti dal backend.
- Form contatti e form admin validati lato client prima dell'invio.
- Uso di escapeHtml per inserire testo dinamico nel DOM riducendo il rischio XSS.

5. Architettura backend

Il backend è basato su Express. Il file server.js configura middleware, rotte pubbliche, rotte admin, protezione JWT e gestione errori. Il file db.js crea un connection pool MySQL con mysql2/promise.

```
public/ -> pagine statiche HTML, CSS, JS, assets
/api -> API pubbliche e admin
sql/ -> schema database e dati iniziali
scripts/ -> utilità per creare o aggiornare admin
```

6. Database MySQL

Tabella	Scopo
projects	Archivia i progetti pubblicati nel portfolio
skills	Archivia competenze e livello indicativo
messages	Archivia i messaggi inviati dal form contatti
admin_users	Archivia admin con password_hash e password_salt

Tutte le query che ricevono dati esterni usano prepared statement con placeholder, in modo da separare struttura SQL e valori inviati dall'utente.

7. API implementate

API pubbliche

```
GET /api/health
GET /api/projects
GET /api/projects/:id
GET /api/skills
POST /api/messages
```

API admin protette

```
POST /api/admin/login
GET /api/admin/me
POST /api/admin/logout
GET /api/admin/projects
GET /api/admin/projects/:id
POST /api/admin/projects
PUT /api/admin/projects/:id
DELETE /api/admin/projects/:id
```

8. Autenticazione e sicurezza

Il login admin segue un flusso stateless con JWT. Il token è firmato lato server e contiene soltanto adminId, email e role. La password non compare mai nel payload.

```
1. Admin invia email/password
2. Server legge admin_users
3. Verifica password con salt + hash PBKDF2
4. Genera JWT firmato con JWT_SECRET
5. Invia cookie admin_token HttpOnly
6. Middleware verifica token e ruolo admin
```

- Cookie HttpOnly: il token non è leggibile da JavaScript.
- SameSite=Lax: riduce il rischio CSRF nei casi comuni.
- Secure in produzione: cookie inviato solo via HTTPS.
- Nessun JWT in localStorage e nessun JWT in query string.
- Pagine admin protette anche lato server con redirect al login.
- Validazione server-side e status code coerenti.

9. Palette e stile visivo

Lo stile è minimale ed elegante, con colori caldi, ampio spazio bianco, card arrotondate, bordi sottili e ombre leggere. Il font principale è Arial, Helvetica, sans-serif.

Elemento	Colori principali
Tema chiaro	#F7F3EC, #FFFDF8, #26211D, #C96442
Tema scuro	#1F1B18, #29231F, #F5EEE5, #E38361
Feedback	Successo #2F6F4E / Errore #A53A31

10. Checklist consegna

- Importare sql/schema.sql e verificare GET /api/health.
- Testare Home, Progetti, Dettaglio, Profilo e Contatti.
- Accedere con admin@example.com / Admin123! e provare CRUD progetti.
- Verificare che admin-dashboard.html senza login reindirizzi al login.
- Consegnare package-lock.json e .env.example, non node_modules e non .env.
- Personalizzare Nome Cognome, email, GitHub, LinkedIn e CV prima della consegna definitiva.

11. Conclusione

Il progetto finale dimostra la realizzazione di un'applicazione web completa: frontend responsive, interazione DOM, comunicazione asincrona con fetch, backend Express, database MySQL, prepared statement e autenticazione JWT per l'area admin. La struttura è pensata per essere consegnabile, testabile e spiegabile all'esame.